

explain me time module

## Python time Module

The Python `time` **module** is used when you need to work with **time, delays, timestamps, and measuring execution time**.

For your Data Engineering / Spark / streaming path, `time` is useful for things like:

- Creating data periodically
- Simulating streaming events
- Waiting between Kafka messages
- Measuring how long code takes
- Getting Unix timestamps
- Building simple producers

---

### 1. Import the module

```
</> Python
```

[▶ Run](#)

```
import time
```

---

### 2. `time.time()`

Returns the current time as a **Unix timestamp**.

```
</> Python
```

[▶ Run](#)

```
import time
```

```
timestamp = time.time()
```

```
print(timestamp)
```

Example:

```
1788024305.12345
```



This number represents the number of seconds since:

```
January 1, 1970 00:00:00 UTC
```



## Why is this useful?

You can use it to record when something happened.

```
</> Python
```



▶ Run

```
import time

event = {
    "event": "login",
    "timestamp": time.time()
}

print(event)
```

Output:

```
</> Python
```



▶ Run

```
{
    'event': 'login',
    'timestamp': 1788024305.12345
}
```

This is useful when creating JSON data for Spark Streaming.

## 3. `time.sleep()`

This is probably the most important function for your streaming practice.

It **pauses your Python program** for a specified number of seconds.

```
</> Python
```



▶ Run

```
import time

print("Start")

time.sleep(3)

print("End")
```

The program waits 3 seconds:

```
Start
... 3 seconds ...
End
```



You can use decimals:

```
</> Python

time.sleep(0.5)
```



▶ Run

Waits for half a second.

---

## Example: Simulating streaming data

```
</> Python

import time

for i in range(5):
    print("Creating event:", i)

    time.sleep(2)
```



▶ Run

Output:

```
Creating event: 0
(wait 2 seconds)
Creating event: 1
(wait 2 seconds)
Creating event: 2
...
```



This is very useful for practicing **Spark Structured Streaming**.

---

## 4. `time.ctime()`

Converts a Unix timestamp into a readable time.

```
</> Python
import time

print(time.ctime())
```



▶ Run

Example:

```
Sat Aug 29 20:25:30 2026
```



You can also provide a timestamp:

```
</> Python
timestamp = time.time()

print(time.ctime(timestamp))
```



▶ Run

---

## 5. `time.localtime()`

Returns the current local time as a structured object.

```
</> Python
import time

current_time = time.localtime()

print(current_time)
```



▶ Run

You can access individual parts:

```
</> Python

print(current_time.tm_year)
print(current_time.tm_mon)
print(current_time.tm_mday)
print(current_time.tm_hour)
print(current_time.tm_min)
print(current_time.tm_sec)
```



▶ Run

Example:

```
2026
8
29
20
25
30
```



## 6. time.gmtime()

Similar to `localtime()` , but returns **UTC time**.

</> Python



▶ Run

```
import time

utc_time = time.gmtime()

print(utc_time)
```

Difference:

```
localtime() → local time
gmtime()    → UTC time
```



## 7. time.strftime()

This is very useful for creating formatted timestamps.

</> Python



▶ Run

```
import time

current_time = time.localtime()

formatted = time.strftime("%Y-%m-%d %H:%M:%S", current_time)

print(formatted)
```

Output:

2026-08-29 20:25:30



Common format codes:

| Code | Meaning |
|------|---------|
| %Y   | Year    |
| %m   | Month   |
| %d   | Day     |
| %H   | Hour    |
| %M   | Minute  |
| %S   | Second  |

For example:

</> Python



▶ Run

```
time.strftime("%Y-%m-%d")
```

Output:

2026-08-29



Or:

</> Python



▶ Run

```
time.strftime("%H:%M:%S")
```

Output:

20:25:30



## 8. Measuring execution time

You can use `time.time()` to measure how long something takes.

&lt;/&gt; Python



Run

```
import time

start = time.time()

# Code
for i in range(1000000):
    x = i * 2

end = time.time()

print("Execution time:", end - start)
```

Example:

Execution time: 0.08



This means the code took approximately 0.08 seconds.

## 9. time.perf\_counter()

For measuring performance, `perf_counter()` is generally better than `time.time()`.

&lt;/&gt; Python



Run

```
import time

start = time.perf_counter()

for i in range(1000000):
    x = i * 2

end = time.perf_counter()

print(end - start)
```

Use:

&lt;/&gt; Python



Run

```
time.time()
```

when you want a **timestamp**.

Use:

&lt;/&gt; Python



Run

```
time.perf_counter()
```

when you want to **measure execution duration**.

---

## 10. time.sleep() + JSON

This is especially relevant to what you're learning.

You can create JSON events every 2 seconds:

&lt;/&gt; Python



Run

```
import json
import time

for i in range(5):

    event = {
        "id": i,
        "event": "purchase",
        "timestamp": time.time(),
        "amount": 100 + i
    }

    print(json.dumps(event))

    time.sleep(2)
```

Output:

```
{"id": 0, "event": "purchase", "timestamp": 1788024305.12, "amount": 100}
```



2 seconds later:

```
{"id": 1, "event": "purchase", "timestamp": 1788024307.13, "amount": 101}
```



This is a simple way to **simulate events arriving over time**.

---

## 11. time vs datetime

You should understand the difference.



## time

Good for:

```
</> Python

time.sleep(5)
time.time()
time.perf_counter()
```



▶ Run

Mainly:

- delays
- timestamps
- measuring execution
- low-level time operations

## datetime

Good for:

```
</> Python

datetime.now()
datetime.today()
timedelta()
```



▶ Run

Mainly:

- dates
- date arithmetic
- comparing dates
- formatting dates
- event-time processing

For your learning:

```
time
↓
basic time operations
↓
datetime
↓
JSON
↓
file generation
↓
Spark Structured Streaming
↓
Kafka
```



## 12. Important functions to remember

You don't need to memorize the entire `time` module.

Focus on these:

```
</> Python

import time

time.time()
time.sleep()
time.ctime()
time.localtime()
time.gmtime()
time.strftime()
time.perf_counter()
```



▶ Run

### Your priority

| Function                      | Importance for you |
|-------------------------------|--------------------|
| <code>time.sleep()</code>     | ★★★★★★             |
| <code>time.time()</code>      | ★★★★★★             |
| <code>time.strftime()</code>  | ★★★★               |
| <code>time.localtime()</code> | ★★★                |

| Function                         | Importance for you |
|----------------------------------|--------------------|
| <code>time.perf_counter()</code> | ★★★★               |
| <code>time.ctime()</code>        | ★★★                |
| <code>time.gmtime()</code>       | ★★★                |

For **Spark Structured Streaming + Kafka**, the two you should understand especially well are:

</> Python

```
time.sleep()
time.time()
```

Run

because you can use them to **simulate continuously arriving data** and attach timestamps to your generated events.



Is this conversation helpful so far?

ChatGPT can make mistakes. Check important info.